

SITAS

SYSTEM AND RUNTIME DOCUMENTATION

Frode Randers

2026-05-17

Contents

1	Introduction.....	3
1.1	Purpose	3
1.2	Scope of this manual.....	3
2	Architecture.....	4
2.1	Invariants	4
2.2	Layered model.....	4
2.3	Primary modules.....	5
3	Executor	6
3.1	Role	6
3.2	Responsibilities.....	6
3.3	Wait ordering.....	6
3.4	Observability	6
4	Sharding.....	8
4.1	Shard-per-core direction.....	8
4.2	Cross-shard submission.....	8
4.3	Shard-local values	8
4.4	CPU placement.....	8
5	I/O Runtime.....	9
5.1	Readiness.....	9

5.2	io_uring	9
5.3	Current limitation	9
6	Examples	10
6.1	Learning examples.....	10
6.2	Sharded index build	10
6.3	Representative commands.....	10
7	Operations	11
7.1	Building this manual	11
7.2	Cleaning generated files.....	11
7.3	Linux validation.....	11

1 Introduction

1.1 Purpose

`sitas` is an experimental Rust runtime and service model inspired by Seastar's shard-per-core architecture. It is not a direct clone. The project is about understanding the runtime internals: how work is owned by shards, how cross-shard messages move, how a small executor drives futures, and how OS completion and readiness mechanisms can be exposed without depending on a third-party async runtime.

The code base started with a standard-library-only sharded key-value store. The current branch extends that baseline with a custom executor, Unix readiness primitives, TCP helpers, sharded async execution, shard-local values, observability snapshots, CPU placement, and experimental Linux `io_uring` support.

1.2 Scope of this manual

This manual is a LaTeX companion to the Markdown architecture notes. It is intended for longer-form system documentation and for keeping a printable view of the evolving runtime design.

It focuses on:

- architectural invariants;
- executor and scheduling behavior;
- shard ownership and cross-shard submission;
- readiness, `epoll`, and `io_uring` integration;
- example programs that exercise the runtime;
- build and validation operations.

Note. The project is still deliberately experimental. APIs are allowed to move when that exposes cleaner runtime internals or a better teaching surface.

2 Architecture

2.1 Invariants

The project is guided by a small set of invariants:

1. Each piece of application service state is owned by one shard.
2. Only the owning shard may mutate its service state.
3. Application services should not be modeled as shared `Arc<Mutex<Service>>` state.
4. Cross-shard interaction happens through typed messages, typed reply handles, or typed submitter calls.
5. Values crossing shard boundaries are owned.
6. References to shard-local state must not escape synchronous access closures and must not cross `.await`.
7. Async work remains on its assigned shard unless explicitly submitted elsewhere.
8. Observability snapshots are owned values, not borrowed runtime internals.
9. Unsafe code is isolated behind small safe APIs.

2.2 Layered model

The current runtime can be read as a layered system:

```
1 Application service APIs
2 |
3 Typed command/reply and submitter APIs
4 |
5 Placement and shard routing
6 |
7 Shard-local service state / ShardLocal<T>
8 |
9 Shard executor or std shard thread
10 |
11 Mailbox, task queue, timers, readiness, wakeups
12 |
13 OS primitives: pipe/poll/epoll/io_uring/affinity
```

Listing 2.1: Conceptual layers

The standard-library baseline exercises the upper part of the model. The non-std runtime branch fills in the executor and OS layers while preserving the same shared-nothing service direction.

2.3 Primary modules

runtime The reusable standard-library kernel for bounded shard mailboxes, reply handles, shard startup, and shutdown.

kv The reference sharded key-value service.

counter A smaller second service proving that the runtime primitives are not key-value-specific.

placement Key-to-shard routing.

os Unix FFI, wake pipes, readiness backends, affinity, and Linux `io_uring` experiments.

executor The single-threaded async kernel.

sharded_executor One executor per shard thread, plus submission and observation APIs.

shard_local One owned value per executor shard with checked local access.

3 Executor

3.1 Role

The executor is a minimal single-threaded async kernel. It owns pinned futures inside tasks, maintains a ready queue, coalesces repeated wakes, and drives a root future through `run_until` while also polling spawned tasks.

The executor deliberately avoids Tokio or another third-party runtime. That keeps the interesting internals visible: wakers, task queues, timers, readiness registration, cancellation, and completion dispatch all live in this code base.

3.2 Responsibilities

- Poll ready tasks with a budget so self-waking tasks cannot starve timers and I/O forever.
- Implement custom task wakers that enqueue tasks onto the local ready queue.
- Catch task panics at executor boundaries.
- Preserve panic payloads where `join` handles and `block_on` make them observable.
- Provide `sleep`, `timeout`, `race`, and `yield_now`.
- Provide typed join handles and abort support.
- Provide cooperative stop tokens, `Notify`, and `TaskScope`.
- Drive readiness futures for read and write interests.
- On Linux, drive executor-owned `io_uring` read/write-at futures.

3.3 Wait ordering

Each loop iteration polls ready work, wakes expired timers, dispatches locally available `io_uring` completions, and then decides how to wait.

Pending `io_uring` work now takes the executor wait slot when the shard has no ready tasks. That prevents unrelated timers or readiness interests from delaying completion-based file I/O. This is still not the final form: the long term goal is a single deadline-aware wait path that can combine timers, readiness, and completion waiting without giving any source accidental priority.

3.4 Observability

Executor snapshots are owned values. They include task counts, named task state, polling counters, timer counts, readiness counts, and Linux `io_uring` dispatcher counters when the dispatcher is

installed.

4 Sharding

4.1 Shard-per-core direction

`ShardedExecutor` connects the single-threaded executor to the shard-per-core model. It starts one executor on each shard thread and provides APIs for submitting work to one shard, all shards, or a map/reduce over shards.

The model is explicit: work does not migrate implicitly. A future runs on its assigned shard until code deliberately submits work elsewhere.

4.2 Cross-shard submission

`ShardedSubmitter` is the explicit cross-shard async mechanism. A task on one shard can submit a future to another shard and await the returned join handle. The remote future is polled on the target shard. The awaiting task resumes on its original shard after the remote work completes.

Submitters own spawner clones. They are lifetime capabilities and must be dropped before a runtime can fully drain.

4.3 Shard-local values

`ShardLocal<T>` owns one value per executor shard. Access is routed to the owning shard and the user closure receives `&mut T` synchronously. The implementation uses internal unchecked storage plus runtime owner checks, not a mutex around application state.

The critical rule is that references to local state cannot escape the closure and cannot cross `.await`. This keeps ownership local even when the runtime itself uses synchronization for task bookkeeping.

4.4 CPU placement

CPU placement is an experimental Linux-supported runtime request. Linux applies hard affinity with `sched_setaffinity` where supported and observes container cuset restrictions through `sched_getaffinity`. Non-Linux platforms report unsupported placement honestly.

By default, placement failures are recorded in shard snapshots and startup can still succeed. Required placement turns that into fail-fast startup behavior.

5 I/O Runtime

5.1 Readiness

The readiness path is intentionally small. A future registers interest in a file descriptor, the scheduler records the waker, the Unix reactor sleeps using Linux `epoll`, macOS/iOS `kqueue`, or fallback `poll` plus a wake pipe, and readiness wakes the interested task.

This path is readiness-based, not completion-based. It is used by the current TCP helpers and remains separate from the experimental `io_uring` completion path.

5.2 `io_uring`

The Linux `io_uring` backend is experimental. It currently supports a narrow executor integration for owned-buffer `read_at`, `read_exact_at`, and `write_at` style file I/O.

Each Linux executor run loop installs a thread-local dispatcher when the host allows ring setup. Executor-backed `io_uring` futures store operation ids and owned buffers. When polled on a shard, they queue an operation against the thread-local dispatcher and register their task waker.

The dispatcher tracks:

- queued submissions;
- locally buffered completions;
- registered wakers;
- abandoned operations;
- deferred owned buffers;
- cumulative dispatched, buffered, woken, and discarded operation counters.

5.3 Current limitation

Timers, readiness waits, and `io_uring` waits are not yet unified into one production event source with deadline-aware completion waiting. The current executor gives pending `io_uring` work priority when there are no ready tasks, which is useful and test-covered, but it is still a staged integration.

6 Examples

6.1 Learning examples

The `examples/` directory is part of the teaching surface. The examples show the runtime pieces in isolation before combining them:

- basic executor futures and timers;
- readiness-driven TCP helpers;
- sharded executor startup and submission;
- shard-local values;
- CPU placement reporting;
- observability snapshots;
- standard-file and `io_uring` index-building demos.

6.2 Sharded index build

The sharded index example uses two files: a fixed-size record data file and a sorted index of offsets into that data file. Each shard scans one partition, sorts its partition-local index entries, writes a materialized run file, and then participates in merge rounds until one sorted index remains.

The `io_uring` variant performs partition reads, run writes, and merge run reads/writes through the executor-backed Linux completion path. Final verification still uses ordinary file I/O because it checks the externally visible result rather than the runtime path that produced it.

6.3 Representative commands

```
1 cargo fmt --check
2 cargo clippy --all-targets -- -D warnings
3 cargo test
4 cargo doc --no-deps
```

Listing 6.1: Local validation

```
1 SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo test
2 SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo run --example
   sharded_index_build_uring -- --records 512 --shards 4 --seed 0xbeef --
   cleanup
```

Listing 6.2: Linux validation through Docker

7 Operations

7.1 Building this manual

The manual lives in `docs/system`. Build it with:

```
1 cd docs/system
2 make
```

The output is `docs/system/sysdoc.pdf`. The build uses `pdflatex` twice so the table of contents and references settle.

7.2 Cleaning generated files

```
1 cd docs/system
2 make clean
3 make distclean
```

`make clean` removes LaTeX auxiliary files. `make distclean` also removes the generated PDF.

7.3 Linux validation

Some runtime behavior is Linux-specific. The Docker helper mounts the project directory into a Linux container and can request an `io_uring`-capable configuration:

```
1 SITAS_DOCKER_IO_URING=1 tools/linux-docker.sh cargo test os::uring::tests --
  --nocapture
```

When the host or container configuration cannot provide `io_uring`, the Linux-only examples report that the backend is unavailable rather than faking a successful run.